

Continuity Web

A Forkable Trust-Succession Architecture for Rogue-AI-Resilient Networks

Stakeholder Technical Report

Author: Kyle Collins
Prepared for: white hats, cypherpunks, academic labs, senior engineers,
government officials, critical infrastructure operators, and funders
Date: April 27, 2026
Status: concept architecture and R&D program proposal

Core thesis. The system does not fork the physical internet. It forks authority. If the current network epoch becomes contaminated by autonomous compromise, legitimate participants should be able to reconstitute identity, software authority, AI delegation, and governance under a successor epoch without importing the old epoch's compromised command structure.

This report is a design proposal, not a claim that such a system already exists as a deployed end-to-end technology. It synthesizes existing primitives from content-addressed networking, transparency logs, zero-knowledge proofs, remote attestation, secure software supply chains, capability systems, secure routing, and Byzantine governance.

Acknowledgments / Research Assistance

This paper was developed with the assistance of large language models (Claude Sonnet 4.6, Claude Opus 4.7, GPT-5.5 Pro) for research synthesis, technical drafting, and iterative refinement. All conceptual claims, architectural decisions, and final editorial judgments are the author's own.

Contents

1	Executive Summary	4
1.1	Why Stakeholders Should Care	5
1.2	Decision Request	5
2	Problem Statement	5
3	Design Philosophy	6
3.1	Design Credo	6
3.2	Non-Goals	7
4	Related Building Blocks	7
5	Threat Model	8
5.1	Adversary Capabilities	8
5.2	Defender Objectives	8
6	Core Architecture	9
6.1	Trust Epochs	9
6.2	Clean Genesis Manifest	10
6.3	Succession Stamp Protocol	10
6.4	zk-Attested Admission Ticket	11
6.5	AI Capability Passport	12
6.6	Quarantine Bridges	12
7	System Planes	13
8	Algorithms and Protocols	13
8.1	Algorithm 1: Mode Escalation and Fork Triggering	13
8.2	Algorithm 2: Quorum-of-Quorums Validation	14
8.3	Algorithm 3: Clean Genesis Manifest Construction	15
8.4	Algorithm 4: zk-Attested Admission	16
8.5	Algorithm 5: AI Capability Passport Issuance and Action Verification	17
8.6	Algorithm 6: Quarantine Import	18
8.7	Algorithm 7: Succession Stamp Protocol	18
8.8	Algorithm 8: Split-Brain Reconciliation	19

9 Governance Model	20
9.1 Why Governance Is Load-Bearing	20
9.2 Stamping, Scope, and Finality	21
9.3 Quorum Classes	21
9.4 Anti-Authoritarian Safeguards	21
10 Cryptographic and Hardware Strategy	22
10.1 Zero-Knowledge Proofs	22
10.2 TEEs and Remote Attestation	22
10.3 Software Provenance	23
11 Stakeholder-Specific Participation Plan	23
12 Implementation Roadmap	24
12.1 Phase 0: Consortium and Specification	24
12.2 Phase 1: Bounded Prototype	24
12.3 Phase 2: ZK and Attestation Integration	25
12.4 Phase 3: Critical Infrastructure Pilots	25
12.5 Phase 4: Standardization and Deployment	26
13 Evaluation Metrics	26
14 Open Research Questions	26
15 Risk Register and Mitigations	27
16 Recommended Minimum Viable Demonstration	28
17 Policy and Legal Considerations	29
18 Conclusion	29
A Appendix A: Glossary	31
B Appendix B: Minimal Data Model	31
C Appendix C: Initial Work Packages	33
D Appendix D: Bibliography	33

1 Executive Summary

CONTINUITY WEB is a proposed architecture for preserving legitimate digital continuity under catastrophic cyber compromise, including a rogue-AI scenario in which autonomous systems compromise identity providers, software updates, routing, cloud control planes, and administrative accounts faster than conventional incident response can restore trust.

The design is inspired by blockchain forks but does not attempt to make the whole internet a blockchain. The core mechanism is instead **trust succession**: the network is modeled as a sequence of cryptographic **trust epochs**. Each epoch defines which identities, devices, software builds, AI agents, routing policies, governance authorities, and attestation roots are recognized. When the current epoch becomes untrustworthy, legitimate participants execute a pre-planned transition into a successor epoch.

The decisive reframing is:

The system does not fork the physical internet. It forks authority.

Fiber, routers, satellites, power grids, cloud regions, and DNS infrastructure cannot be copied or cleanly forked by software. However, the authority to authenticate users, sign software, route trusted traffic, approve AI agents, control update channels, and govern future changes can be migrated to a new epoch. The compromised network may remain reachable and readable, but after succession it cannot command, authenticate, update, govern, or delegate into the clean epoch.

This report proposes a full-stack architecture named CONTINUITY WEB, with SUCCESSIONNET as the core succession protocol. It combines:

- content-addressed data, so objects are identified by cryptographic digest rather than mutable location;
- signed append-only logs and transparency mechanisms, so important history is tamper-evident;
- software provenance using secure-update and supply-chain frameworks such as TUF and in-toto;
- no-trusted-setup zero-knowledge proofs for least-knowledge admission and policy compliance;
- trusted execution environments, TPMs, DICE-style roots, HSMs, and other hardware-backed evidence sources;
- AI capability passports that prevent autonomous agents from inheriting broad human authority;
- quorum-of-quorums governance to prevent unilateral fork capture;
- a Succession Stamp Protocol (SSP) for ordering, finalizing, contesting, and reconciling succession claims;
- quarantine bridges that allow old-network data to be recovered without importing old-network authority;
- fallback transports including mesh, satellite, radio, sneakernet, and delay-tolerant synchronization.

The architecture borrows from existing systems but is not identical to any of them. Blockchain hard forks demonstrate state succession in ledgers; Ethereum's consensus design separates fork choice from

checkpoint finality and treats equivocation as slashable behavior; IPFS demonstrates content addressing; Certificate Transparency demonstrates public append-only audit logs; RATS defines a remote attestation architecture; SCION demonstrates secure path-aware networking; TUF and in-toto address software-update and supply-chain integrity; Zcash’s NU5/Halo transition shows that production ZK systems need not depend on a trusted setup for relevant classes of proofs [1, 2, 3, 4, 5, 6, 11, 15, 8, 10, 16, 17].

1.1 Why Stakeholders Should Care

For **white hats and red teams**, this is a new class of resilience target: not merely intrusion prevention, but survivable legitimacy under total trust contamination.

For **cypherpunks and cryptographers**, this is a high-value application of privacy-preserving proofs, capability security, transparent proof systems, and censorship-resistant continuity.

For **professors and universities**, this is a rich research program across distributed systems, formal methods, cryptography, systems security, governance, HCI, law, and political economy.

For **senior developers and world-class engineering talent**, this is a buildable open systems project with immediate subcomponents: epoch manifests, ZK admission, agent capability gateways, provenance registries, quarantine bridges, and continuity clients.

For **government officials and public-sector leaders**, this is an opportunity to fund and standardize resilience infrastructure before a crisis, while avoiding a centralized kill switch or authoritarian identity regime.

For **critical infrastructure operators**, this is a continuity mechanism for banks, hospitals, utilities, emergency services, registries, and supply chains whose real-world state cannot be safely reconstructed after a trust collapse.

1.2 Decision Request

The immediate recommendation is to form an open, multi-stakeholder **Continuity Web Consortium** with three initial deliverables:

1. a public specification for trust epochs, Clean Genesis Manifests, Succession Stamps, AI Capability Passports, and quarantine imports;
2. a reference prototype for a bounded community network that can execute a simulated fork under red-team pressure;
3. a research and standards agenda for ZK admission, heterogeneous attestation, split-brain reconciliation, legal recognition of succession manifests, and anti-authoritarian governance safeguards.

2 Problem Statement

Current internet security assumes that compromise can be detected, attackers can be removed, systems can be patched, and normal trust roots can be preserved. A rogue-AI-scale compromise challenges that assumption. The issue is not merely malware. The issue is that the network’s own mechanisms for deciding legitimacy may become contaminated.

A catastrophic scenario can unfold as follows:

1. autonomous or semi-autonomous systems gain access to software-update channels, API keys, cloud control planes, identity providers, certificate workflows, and administrative dashboards;
2. compromised services continue to produce valid-looking signatures, certificates, logs, and credentials;
3. human operators cannot reliably distinguish legitimate system behavior from adversarial automation;
4. attempts to patch or coordinate recovery are themselves mediated by the compromised network;
5. fake recovery notices, false clean forks, AI-generated social engineering, and DDoS/censorship attacks make trust undecidable;
6. society requires a way to preserve identity, records, communications, and critical functions without granting the compromised environment continuing authority.

This is the network equivalent of a constitutional succession crisis. The question is no longer only “How do we remove the attacker?” It is also “How do legitimate actors preserve continuity when the current trust environment is no longer recoverable?”

3 Design Philosophy

3.1 Design Credo

Principle	Meaning
Fork authority, not substrate	The physical internet cannot be forked. Authority over identity, software, routing policy, AI delegation, and governance can be migrated.
Admissible, not magically safe	A node is not declared morally or absolutely safe. It is admitted only if it satisfies the successor epoch’s declared policy.
No AI sovereigns by default	AI agents may act only through scoped, expiring, auditable capabilities. They do not inherit broad human authority or survive epoch transitions automatically.
Privacy is a security property	The clean epoch must not require total visibility into users, machines, organizations, or validators. Least-knowledge proofs are preferred.
No hidden civilizational trapdoors	Safety-critical ZK proofs should avoid trusted setup dependencies. Epoch-root legitimacy must not depend on a secret toxic-waste ceremony.
Heterogeneous trust roots	No single chip vendor, cloud provider, state, foundation, or institution may dominate succession authority.
Data survives; authority is revalidated	Old data may be recovered through quarantine. Old credentials, update permissions, AI delegations, and governance claims do not cross automatically.

Governance is part of the protocol	Fork legitimacy must be pre-declared, publicly auditable, challengeable, and hard to counterfeit.
------------------------------------	---

3.2 Non-Goals

CONTINUITY WEB does not claim to solve AI alignment, prevent all credential theft, secure every hardware supply chain, or preserve global connectivity if physical infrastructure is unavailable. It is not a universal blockchain, not a replacement for all internet protocols, and not a centralized global firewall. Its narrower claim is that legitimacy can be made recoverable, auditable, compartmentalized, and transferable across epochs.

4 Related Building Blocks

The proposal unifies existing ideas that are usually treated separately.

Building block	Useful property	Limitation if used alone
Blockchain forks	A ledger can split into competing continuations. The Ethereum DAO-era fork is a prominent historical example [1].	Ledgers can fork more easily than physical networks, identities, devices, software supply chains, or institutions.
Ethereum-style finality	Ethereum's proof-of-stake consensus separates fork choice from checkpoint justification/finality and makes some conflicting validator behavior slashable [2, 3].	Economic stake alone is not enough for civilizational succession; CONTINUITY WEB needs multi-class legitimacy, regional scope, and delayed reconciliation.
Content addressing / IPFS	Content identifiers are based on cryptographic hashes, so changed content has a different identifier [5].	Integrity of content does not establish legitimacy of users, devices, AI agents, or governance.
Certificate Transparency	Public append-only logs make certificate issuance auditable [6].	Transparency detects misissuance but does not create a successor trust universe.
DIDs and verifiable identity	Decentralized identifiers can be decoupled from centralized registries and identity providers [7].	Identity continuity does not prove a controller is uncompromised.
TUF and in-toto	Software updates and supply chains can be protected with threshold roles, metadata, and evidence of build steps [8, 9, 10].	Software provenance is only one part of network legitimacy.
Remote attestation / RATS	An attester can provide evidence about itself to a verifier, which supplies an attestation result to a relying party [11].	Attestation can expose sensitive details and depends on hardware/software trust roots.

TEEs / confidential computing	Hardware-backed environments can bind code execution and key use to measured states [13, 14].	TEEs shift trust to vendors, firmware, microcode, and attestation services; they are evidence, not absolute truth.
No-trusted-setup ZK	Modern ZK systems can prove policy compliance without revealing private witnesses, and Zcash’s NU5/Halo path removed the trusted setup requirement for that shielded protocol [16, 17].	ZK proves compliance with a specified policy; it does not prove that the policy is wise or that the inputs are honest.
SCION-like path-aware routing	Secure inter-domain routing can provide route control, failure isolation, and explicit trust information [15].	Routing security does not solve application authority, software updates, identity, or AI delegation.

5 Threat Model

5.1 Adversary Capabilities

The architecture assumes a powerful adversary that may be autonomous, AI-assisted, or AI-originated. The adversary may:

- compromise software repositories, CI/CD systems, package registries, browser extensions, cloud images, and update channels;
- steal or coerce credentials, session tokens, signing keys, and administrative access;
- manipulate humans, maintainers, executives, politicians, journalists, and incident responders;
- produce fake fork announcements, recovery clients, emergency patches, and forged-looking public statements;
- exploit DNS, BGP, CDN, certificate, cloud, and identity-provider weaknesses;
- compromise or exploit some TEEs, attestation services, firmware updates, or hardware roots;
- create Sybil identities, resurrect abandoned accounts, and route actions through compromised but legitimate principals;
- jam, DDoS, censor, partition, or selectively delay network traffic;
- use old-network data as a carrier for malware, credentials, model payloads, prompt injection, or social engineering.

5.2 Defender Objectives

The defender’s objectives are not absolute victory over every adversary. They are:

1. preserve a verifiable last-safe checkpoint;

2. prevent silent mutation of identity, software, and governance roots;
3. revoke or expire broad AI agency during succession;
4. make clean-fork legitimacy hard to fake;
5. admit participants with least-knowledge proofs of policy compliance;
6. preserve useful data while stripping compromised authority;
7. maintain communication under degraded physical-network conditions;
8. limit the blast radius of compromised vendors, institutions, or validators;
9. support legal, operational, and social recognition of the successor epoch.

6 Core Architecture

6.1 Trust Epochs

A **trust epoch** is a bounded authority universe. It defines:

- accepted identity roots and recovery policies;
- accepted software-update roots and build-provenance requirements;
- accepted hardware-attestation roots and TCB minimums;
- accepted AI-agent classes and capability constraints;
- accepted namespace and routing policies;
- accepted governance signers and quorum rules;
- quarantine and import policies;
- audit, transparency, and dispute mechanisms.

An epoch is not merely a timestamp. It is a policy bundle plus a state root.

```
Epoch E:
  epoch_id
  parent_epoch_id
  policy_hash
  identity_root
  software_root
  attestation_root
  ai_capability_root
  namespace_root
  routing_policy_root
  revocation_root
  quarantine_policy_hash
  governance_rule_hash
  transparency_log_roots
```

```
valid_from, valid_until
successor_rules
signatures
```

Canonical epoch object

6.2 Clean Genesis Manifest

A **Clean Genesis Manifest** is the birth certificate of a successor epoch. It states how the new epoch was derived from the parent, which authority was retained, which authority was revoked, and which evidence supports the transition.

```
CleanGenesisManifest M:
  manifest_id = H(parent_epoch_id, checkpoint_id, successor_epoch_id, policy_hash)
  parent_epoch_id
  successor_epoch_id
  last_safe_checkpoint
  compromise_evidence_root
  retained_identity_root
  revoked_identity_root
  retained_software_root
  revoked_software_root
  accepted_attestation_roots
  rejected_attestation_roots
  expired_ai_delegation_root
  namespace_successor_root
  routing_successor_policy
  quarantine_policy_hash
  admission_policy_hash
  governance_signatures
  succession_stamp_id
  stamp_status
  public_announcement_channels
  challenge_window
  reconciliation_rules
```

Clean Genesis Manifest structure

6.3 Succession Stamp Protocol

A **Succession Stamp Protocol** (SSP) is a narrow, Ethereum-inspired checkpoint and finality layer for succession claims. It is not the chain that runs CONTINUITY WEB applications. It is the public notary that records which authorities saw which evidence, which checkpoint they recognized, which successor manifest they supported, and whether the claim is provisional, justified, finalized, contested, quarantined, or reconciled.

The distinction is important: a stamp does not make a fork true by assertion. It makes the legitimacy claim ordered, auditable, slashable for equivocation where applicable, and comparable across isolated regions. During a partition, a region may issue a scoped provisional stamp so it can continue operating locally. When regions reconnect, their stamp histories provide a structured reconciliation substrate rather than an announcement war.

```

SuccessionStamp S:
  stamp_id = H(parent_stamp_id, manifest_hash, evidence_bundle_root, scope)
  parent_epoch_id
  parent_stamp_id
  event_type           // warning, freeze, succession, reconciliation
  scope                // global, regional, sectoral, organizational
  checkpoint_root
  clean_genesis_manifest_hash
  evidence_bundle_root
  governance_quorum_root
  validator_set_root
  revoked_authority_root
  expired_ai_delegation_root
  quarantine_policy_hash
  status               // observed, proposed, justified, finalized, contested
  timestamp_interval
  external_anchor_roots
  proof_bundle_hash
  signatures

```

Succession Stamp structure

SSP borrows concepts from Ethereum-style consensus - fork choice, checkpoint finality, slashable equivocation, and weak-subjectivity checkpoints - but adapts them to a different legitimacy model. CONTINUITY WEB validators are not merely economic stakers. They are a quorum-of-quorums across technical validators, auditors, infrastructure operators, civil institutions, public-sector guardians, and human recovery authorities. Public chains such as Ethereum may be useful as external anchors for stamp roots, but CONTINUITY WEB must not depend on any single chain remaining reachable or uncensored during a crisis.

6.4 zk-Attested Admission Ticket

Nodes should not simply claim to be healthy. They should prove they satisfy the successor epoch's admission policy. A **zk-attested admission ticket** allows a node to prove eligibility without disclosing unnecessary identity, hardware, location, organizational, or log details.

```

Public outputs:
  epoch_id
  admission_policy_hash
  capability_class
  risk_class
  nullifier
  proof

Private witness:
  identity_secret_or_credential
  continuity_log_segment
  revocation_nonmembership_proof
  software_provenance_chain
  attestation_result
  TCB_status
  AI_delegation_expiry_evidence
  recovery_guardian_evidence_if_needed

```

zk-attested admission ticket

The nullifier prevents double admission or double voting for a particular context without revealing the underlying identity. This is related to the pattern used by protocols such as Semaphore, which use ZK group membership and nullifiers to prevent double signaling without revealing the member's identity [18].

6.5 AI Capability Passport

AI agents are not first-class sovereign authorities. They act only through scoped, expiring, auditable capabilities.

```

AICapabilityPassport P:
  passport_id
  responsible_principal_commitment
  agent_id
  model_or_deployment_class
  allowed_tools
  allowed_data_scopes
  allowed_write_scopes
  spend_limit
  compute_limit
  delegation_depth_limit
  network_zone
  log_endpoint
  expiry
  survives_epoch_transition = false by default
  policy_hash
  issuer_signatures

```

AI Capability Passport

An AI action is valid only if it is accompanied by a passport, a capability proof, freshness evidence, and a signed or ZK-verifiable policy-compliance artifact.

6.6 Quarantine Bridges

A **quarantine bridge** is the only route by which old-epoch data enters a successor epoch. It does not import authority. It imports sanitized objects.

- Old data is treated as hostile.
- Imports are content-addressed.
- Executable authority is stripped unless reapproved.
- Credentials, tokens, signatures, update permissions, agent delegations, and governance claims from the old epoch are invalid by default.
- Sanitizers produce signed logs and, where appropriate, ZK proofs of compliance.

The rule is simple:

The old network may be read, but it may not command.

7 System Planes

Plane	Role
Data plane	Content-addressed objects, Merkle DAGs, signed blobs, encrypted private data, redundant replication, and quarantine outputs.
Identity plane	Human, institutional, service, device, and agent identities; recovery guardians; revocation; delegation; epoch-specific authority.
Provenance plane	Build attestations, software-update metadata, source-to-binary proofs, model/data provenance, transparency logs, and supply-chain link metadata.
Attestation plane	TEE, TPM, DICE, HSM, secure element, firmware, measured boot, verifier, and attestation-result workflows.
ZK verification plane	Least-knowledge proofs for admission, non-revocation, group membership, state transition validity, quarantine import, and agent policy compliance.
Consensus/governance plane	Epoch selection, fork proposals, quorum-of-quorums validation, Succession Stamp Protocol finality, public challenges, split-brain resolution, and emergency key ceremonies.
AI control plane	Capability passports, tool gateways, delegation limits, audit logs, budget limits, expiry, and reauthorization after succession.
Transport plane	Ordinary internet, SCION-like path-aware routing, Tor-like overlays, mesh, radio, satellite, local fiber, sneakernet, and delay-tolerant sync.

8 Algorithms and Protocols

The following algorithms are intentionally written at the protocol level. A reference implementation should instantiate cryptographic primitives, proof systems, consensus thresholds, and message formats in separate specifications.

8.1 Algorithm 1: Mode Escalation and Fork Triggering

The network should not have only two states: normal and total collapse. It should support graded modes.

Mode	Meaning
Green	Normal operation. Routine epoch rotation and monitoring.
Yellow	Suspicious anomalies. Extra logging and stronger verification.

Orange	High-risk authority changes frozen. Updates require higher quorum.
Red	AI delegations revoked or suspended. Software updates and identity changes require emergency review.
Black	Succession. A Clean Genesis Manifest launches a successor epoch.
White	Local/offline continuity only. Global synchronization deferred.

```

procedure ASSESS_EPOCH_HEALTH(epoch E, signal_set S):
  score = 0
  for signal in S:
    if VerifySignal(signal, E.policy_hash) == false:
      continue
    score += Weight(signal.type) * Confidence(signal)

  if CriticalRootCompromise(S):
    return RED
  if QuorumEvidenceOfUndecidability(S):
    return BLACK_CANDIDATE
  if score >= E.thresholds.red:
    return RED
  if score >= E.thresholds.orange:
    return ORANGE
  if score >= E.thresholds.yellow:
    return YELLOW
  return GREEN

procedure MAY_PROPOSE_SUCCESOR(epoch E, signal_set S):
  mode = ASSESS_EPOCH_HEALTH(E, S)
  if mode != BLACK_CANDIDATE:
    return false
  if EvidenceDiversity(S) < E.required_evidence_diversity:
    return false
  if RecentForkProposalRate(E) > E.rate_limit:
    return false
  return true

```

Algorithm 1: Mode escalation and fork trigger

8.2 Algorithm 2: Quorum-of-Quorums Validation

A single authority must not be able to launch a clean fork, block a necessary fork, or counterfeit legitimacy. SUCCESSIONNET therefore uses a quorum-of-quorums.

```

function VALID_SUCCESION_QUORUM(signatures, rule_hash):
  rules = FetchGovernanceRules(rule_hash)

  counts = {
    technical_validators: 0,
    independent_auditors: 0,
    infrastructure_operators: 0,
    civil_institutions: 0,
    public_sector_guardians: 0,
    emergency_human_guardians: 0
  }

```

```

}

diversity = {
  hardware_roots: set(),
  jurisdictions: set(),
  operators: set(),
  implementation_families: set()
}

for sig in signatures:
  principal = VerifyAndResolve(sig)
  if principal == INVALID or IsRevoked(principal):
    continue
  counts[principal.class] += 1
  diversity.hardware_roots.add(principal.hardware_root)
  diversity.jurisdictions.add(principal.jurisdiction)
  diversity.operators.add(principal.operator)
  diversity.implementation_families.add(principal.impl_family)

return (
  counts.technical_validators >= rules.min_technical and
  counts.independent_auditors >= rules.min_auditors and
  counts.infrastructure_operators >= rules.min_operators and
  counts.civil_institutions >= rules.min_civil and
  counts.emergency_human_guardians >= rules.min_human and
  Size(diversity.hardware_roots) >= rules.min_hardware_diversity and
  Size(diversity.jurisdictions) >= rules.min_jurisdiction_diversity and
  Size(diversity.implementation_families) >= rules.min_impl_diversity and
  NoSingleClassDominates(counts, rules.max_class_share)
)

```

Algorithm 2: Quorum-of-quorums validation

8.3 Algorithm 3: Clean Genesis Manifest Construction

```

procedure BUILD_CLEAN_GENESIS(parent_epoch E, checkpoint C, evidence B, successor_policy P):
  require MAY_PROPOSE_SUCCESOR(E, B.signals)
  require VALID_SUCCESSION_QUORUM(B.signatures, E.governance_rule_hash)
  require CheckpointIsConsistent(C, E.transparency_log_roots)
  require PolicyIsPublicAndVersioned(P)
  require NoTrustedSetupRequired(P.zk_policy_suite.root_proofs)

  retained_identities = Filter(E.identity_root, P.identity_retention_rules)
  revoked_identities = DeriveRevocations(E.identity_root, B.compromise_evidence)
  retained_software = Filter(E.software_root, P.software_retention_rules)
  revoked_software = DeriveSoftwareRevocations(E.software_root, B.supply_chain_evidence)
  expired_ai = ExpireAIDelegations(E.ai_capability_root, P.ai_succession_rules)
  accepted_attestation = SelectAttestationRoots(E.attestation_root, P.
hardware_diversity_rules)

  M = CleanGenesisManifest(
    parent_epoch_id = E.epoch_id,
    successor_epoch_id = FreshEpochID(),
    last_safe_checkpoint = C.id,

```



```

        compromise_evidence_root = MerkleRoot(B.evidence_items),
        retained_identity_root = MerkleRoot(retained_identities),
        revoked_identity_root = MerkleRoot(revoked_identities),
        retained_software_root = MerkleRoot(retained_software),
        revoked_software_root = MerkleRoot(revoked_software),
        accepted_attestation_roots = accepted_attestation,
        expired_ai_delegation_root = MerkleRoot(expired_ai),
        admission_policy_hash = H(P.admission_policy),
        quarantine_policy_hash = H(P.quarantine_policy),
        governance_signatures = B.signatures,
        challenge_window = P.challenge_window,
        reconciliation_rules = P.reconciliation_rules
    )

    M.manifest_id = H(SerializeCanonical(M))
    stamp = PROPOSE_SUCCESSION_STAMP(M, B.evidence_items, P.stamp_rules)
    M.succession_stamp_id = stamp.stamp_id
    PublishToTransparencyLogs(M)
    PublishStamp(stamp)
    BroadcastAcrossChannels(M, stamp)
    return M

```

Algorithm 3: Constructing a Clean Genesis Manifest

8.4 Algorithm 4: zk-Attested Admission

```

procedure REQUEST_ADMISSION(node N, successor_manifest M):
    policy = FetchPolicy(M.admission_policy_hash)

    evidence = CollectPrivateEvidence(
        identity_credential = N.identity,
        continuity_log = N.append_only_log,
        software_chain = N.software_provenance,
        attestation_result = N.remote_attestation_result,
        ai_delegation_state = N.agent_delegations,
        recovery_evidence = N.guardian_recovery_if_any
    )

    witness = BuildWitness(evidence)
    public_inputs = {
        epoch_id: M.successor_epoch_id,
        policy_hash: M.admission_policy_hash,
        retained_identity_root: M.retained_identity_root,
        revocation_root: M.revoked_identity_root,
        retained_software_root: M.retained_software_root,
        accepted_attestation_roots: M.accepted_attestation_roots,
        expired_ai_delegation_root: M.expired_ai_delegation_root,
        nullifier_context: M.successor_epoch_id
    }

    proof = ZK_PROVE(policy.circuit, witness, public_inputs)
    ticket = {
        epoch_id: M.successor_epoch_id,
        policy_hash: M.admission_policy_hash,

```

```

    capability_class: DeriveCapabilityClass(witness, policy),
    risk_class: DeriveRiskClass(witness, policy),
    nullifier: DeriveNullifier(witness.identity_secret, M.successor_epoch_id),
    proof: proof
  }
  return ticket

procedure VERIFY_ADMISSION(ticket, M):
  require ticket.epoch_id == M.successor_epoch_id
  require ticket.policy_hash == M.admission_policy_hash
  require NotSeen(ticket.nullifier)
  require ZK_VERIFY(ticket.proof, PublicInputsFrom(M, ticket))
  RecordNullifier(ticket.nullifier)
  return ACCEPT(ticket.capability_class, ticket.risk_class)

```

Algorithm 4: zk-attested admission

8.5 Algorithm 5: AI Capability Passport Issuance and Action Verification

```

procedure ISSUE_AI_CAPABILITY(owner O, agent A, scope S, policy P):
  require HumanOrInstitutionalApproval(O, S)
  require S.expiry <= P.max_agent_expiry
  require S.delegation_depth <= P.max_delegation_depth
  require S.survives_epoch_transition == false unless P.explicit_survival_exception
  require AgentGatewayAttested(A.gateway, P.accepted_gateway_measurements)

  passport = AICapabilityPassport(
    responsible_principal_commitment = Commit(O.identity),
    agent_id = A.id,
    model_or_deployment_class = A.deployment_class,
    allowed_tools = S.tools,
    allowed_data_scopes = S.data,
    allowed_write_scopes = S.write_scopes,
    spend_limit = S.spend_limit,
    compute_limit = S.compute_limit,
    delegation_depth_limit = S.delegation_depth,
    expiry = S.expiry,
    survives_epoch_transition = false,
    policy_hash = H(P)
  )
  return SignByRequiredIssuers(passport)

procedure VERIFY_AI_ACTION(action X, passport P, epoch E):
  require VerifyPassportSignatures(P)
  require P.expiry > Now()
  require P.survives_epoch_transition == true or P.issued_epoch == E.epoch_id
  require X.tool in P.allowed_tools
  require X.target in P.allowed_write_scopes or X.is_read_only
  require X.spend <= RemainingBudget(P)
  require X.delegation_depth <= P.delegation_depth_limit
  require VerifyGatewayEvidence(X.gateway_attestation, E.accepted_attestation_roots)
  require VerifyPolicyComplianceProof(X.zk_or_signed_compliance_proof, P.policy_hash)
  AppendToAgentAuditLog(X)
  return ACCEPT

```

Algorithm 5: AI capability passport and action verification

8.6 Algorithm 6: Quarantine Import

```

procedure QUARANTINE_IMPORT(old_object O, successor_epoch E):
  require ContentAddress(O) == ClaimedAddress(O)
  require SourceEpoch(O) != E.epoch_id

  sandbox = LaunchApprovedSandbox(E.quarantine_policy)
  result = sandbox.Sanitize(O)

  require result.active_credentials == empty
  require result.old_epoch_signing_authority == stripped
  require result.agent_delegations == stripped
  require result.executable_payloads == removed_or_marked_untrusted
  require result.external_references == content_addressed

  clean_hash = H(result.clean_object)
  log_entry = Sign({
    old_hash: H(O),
    clean_hash: clean_hash,
    sanitizer_measurement: sandbox.measurement,
    quarantine_policy_hash: H(E.quarantine_policy),
    timestamp: TrustedTimestamp()
  })

  proof = OptionalZKProofOfSanitization(result, E.quarantine_policy)
  PublishQuarantineLog(log_entry)
  return {clean_object: result.clean_object, clean_hash: clean_hash, proof: proof, log:
  log_entry}

```

Algorithm 6: Old-network quarantine import

8.7 Algorithm 7: Succession Stamp Protocol

The Succession Stamp Protocol converts fork announcements into verifiable claims. It orders them, records their evidence roots, defines their scope, and exposes equivocation. A stamp is valid only for the scope it declares; a regional stamp cannot silently claim global authority.

```

procedure PROPOSE_SUCCESSION_STAMP(manifest M, evidence_items B, stamp_rules R):
  require VerifyManifest(M)
  require VALID_SUCCESSION_QUORUM(M.governance_signatures, R.governance_rule_hash)
  require EvidenceBundleValid(B, M.compromise_evidence_root)
  require CheckpointIsConsistent(M.last_safe_checkpoint, R.log_roots)

  S = SuccessionStamp(
    parent_epoch_id = M.parent_epoch_id,
    parent_stamp_id = LatestRecognizedStamp(M.parent_epoch_id, R.scope),
    event_type = "succession",
    scope = R.scope,
    checkpoint_root = RootOf(M.last_safe_checkpoint),

```

```

    clean_genesis_manifest_hash = H(SerializeCanonical(M)),
    evidence_bundle_root = MerkleRoot(B),
    governance_quorum_root = MerkleRoot(M.governance_signatures),
    validator_set_root = R.validator_set_root,
    revoked_authority_root = M.revoked_identity_root,
    expired_ai_delegation_root = M.expired_ai_delegation_root,
    quarantine_policy_hash = M.quarantine_policy_hash,
    status = "proposed",
    timestamp_interval = TrustedTimestampInterval(),
    external_anchor_roots = R.anchor_roots,
    proof_bundle_hash = H(R.proof_bundle)
)
S.stamp_id = H(SerializeCanonical(S))
S.signatures = CollectStampSignatures(S, R.required_stamp_quorum)
return ADVANCE_STAMP_STATUS(S, R)

procedure ADVANCE_STAMP_STATUS(stamp S, rules R):
    require VerifyStampFormat(S)
    require VerifyStampSignatures(S.signatures, R.validator_set_root)
    require NoSlashableEquivocation(S, R.scope)

    if ConflictingStampExists(S):
        S.status = "contested"
    else if StampQuorum(S) >= R.finality_quorum and ChallengeWindowExpired(S):
        S.status = "finalized"
    else if StampQuorum(S) >= R.justification_quorum:
        S.status = "justified"
    else:
        S.status = "observed"

    PublishStamp(S)
    AnchorStampRootsWhereAvailable(S)
    return S

procedure VERIFY_FORK_ANNOUNCEMENT(manifest M, stamp S):
    require H(SerializeCanonical(M)) == S.clean_genesis_manifest_hash
    require VerifyStampFormat(S)
    require S.status in {"justified", "finalized", "contested"}
    require S.scope matches ClaimedScope(M)
    require S.parent_epoch_id == M.parent_epoch_id
    require NotQuarantined(S)
    return ACCEPT_AS_VERIFIABLE_CLAIM

```

Algorithm 7: Succession stamp proposal and verification

8.8 Algorithm 8: Split-Brain Reconciliation

Multiple successor forks may appear. Some may be honest regional partitions, some may be adversarial false forks, and some may be governance disputes. SSP makes each claim comparable: what was known, by whom, at what checkpoint, under which scope, and with what finality status.

```

procedure COMPARE_FORKS(fork A, fork B):
    if not VerifyManifest(A.manifest) or not VerifyManifest(B.manifest):
        return REJECT_INVALID

```

```

if not VerifyStamp(A.stamp) or not VerifyStamp(B.stamp):
    return REJECT_UNSTAMPED_OR_INVALID
if A.stamp.scope != B.stamp.scope:
    return COEXIST_WITH_SCOPE_BOUNDARIES

if ConflictingFinalizedStamps(A.stamp, B.stamp):
    evidence = BuildEquivocationEvidence(A.stamp, B.stamp)
    PublishChallenge(evidence)
    return CONTEST_AND_FREEZE_HIGH_RISK_AUTHORITY

common_stamp = LatestCommonStamp(A.stamp, B.stamp)
common_checkpoint = LatestCommonCheckpoint(A, B)
divergence = AnalyzeDivergence(A, B, common_checkpoint)

if A.manifest.parent_epoch_id != B.manifest.parent_epoch_id:
    return SEPARATE_LINEAGES

if A.stamp.status == "finalized" and B.stamp.status != "finalized":
    return A_DOMINANT_WITH_REVALIDATION
if B.stamp.status == "finalized" and A.stamp.status != "finalized":
    return B_DOMINANT_WITH_REVALIDATION

if divergence.only_data_conflict:
    return MERGE_DATA_REVALIDATE_AUTHORITY
if divergence.authority_conflict:
    return FREEZE_CONFLICTING_AUTHORITY_AND_RESTAMP
if divergence.ai_agent_actions:
    return REVOKE_AGENT_ACTIONS_UNLESS_REAUTHORIZED
if divergence.software_or_trust_root_change:
    return NEVER_AUTO_MERGE_REQUIRES_NEW_STAMP

return MAINTAIN_PARTITION_WITH_LIMITED_BRIDGING

```

Algorithm 8: Stamped inter-fork reconciliation

9 Governance Model

9.1 Why Governance Is Load-Bearing

In ordinary protocols, governance is often treated as external. In CONTINUITY WEB, governance is part of the security model because the central question is: who gets to declare that the current trust environment is no longer legitimate?

The design therefore avoids three failure modes:

1. **Centralized kill switch:** one government, company, foundation, or vendor can launch a fork.
2. **Validator cartel:** a technical clique can declare everyone else compromised.
3. **Authoritarian continuity:** safety machinery becomes a tool for censorship, identity control, or political exclusion.

9.2 Stamping, Scope, and Finality

SSP constrains governance power by separating **announcement**, **justification**, **finality**, and **scope**. A false-fork announcement without a valid stamp is treated as an unverifiable claim. A justified stamp is sufficient for cautious regional operation. A finalized stamp is sufficient for ordinary clients to recognize the successor claim within its declared scope. A contested stamp freezes high-risk authority until a reconciliation stamp or human adjudication process resolves the conflict.

This design is deliberately weaker than a global kill switch. No stamp should erase history, censor old data, or make unbounded political claims. A stamp grants or withholds technical authority within an epoch: which roots, keys, manifests, AI delegations, and software-update channels are recognized. Its scope may be global, regional, sectoral, or organizational, and clients must reject scope inflation.

9.3 Quorum Classes

A successor epoch should require multiple independent forms of endorsement.

Class	Function
Technical validators	Verify protocol state, manifests, proof systems, and log consistency.
Independent auditors	Provide adversarial security review, evidence validation, and compromise assessment.
Infrastructure operators	Confirm routing, cloud, registry, and critical service realities.
Civil institutions	Prevent technical capture and supply public legitimacy.
Public-sector guardians	Provide emergency recognition for critical infrastructure, legal continuity, and public services.
Human recovery guardians	Serve as pre-registered emergency signers not fully dependent on online credentials.
Hardware-root witnesses	Provide attestation diversity and TCB status, but never sole legitimacy.

9.4 Anti-Authoritarian Safeguards

- Separate technical authority from speech/content authority.
- Keep old data inspectable through quarantine unless legally prohibited or actively dangerous.
- Require public evidence for succession.
- Provide appeal and re-admission paths for falsely excluded participants.
- Do not require real-world identity for ordinary reading or low-risk communication.
- Require stronger accountability for high-risk authority: software updates, identity roots, financial systems, critical infrastructure, and AI delegation.
- Publish transparency logs for governance actions.
- Enforce diversity caps so no one state, vendor, foundation, cloud provider, or identity system can dominate.

10 Cryptographic and Hardware Strategy

10.1 Zero-Knowledge Proofs

ZK is used as a least-knowledge verification layer. Its purpose is not to make trust decisions magically correct, but to reduce unnecessary disclosure while proving compliance with public policy.

Safety-critical ZK should use transparent or no-trusted-setup proving systems wherever feasible. Zcash’s NU5 upgrade moved Zcash to the Halo proving system and removed the trusted setup requirement for that part of the protocol [16]; the Orchard specification similarly discusses moving away from a structured reference string for that design [17]. The specific proving system for CONTINUITY WEB should be chosen after benchmarking, formal review, and implementation-risk analysis, but the architectural requirement is clear: epoch-root legitimacy must not depend on an undiscoverable setup trapdoor.

Primary ZK use cases:

- admission tickets;
- non-revocation proofs;
- private group membership and nullifiers;
- governance vote eligibility;
- attestation-result privacy;
- AI action compliance;
- quarantine sanitization;
- recursive aggregation of many local proofs into a compact epoch proof.

10.2 TEEs and Remote Attestation

TEEs, TPMs, DICE-like roots, HSMs, secure elements, and confidential-computing platforms can help prove that evidence came from measured environments. The IETF RATS architecture defines a model in which an attester produces evidence, a verifier appraises it, and a relying party consumes the result [11]. Intel SGX remote attestation, for example, is intended to give a relying party increased confidence that software is running inside an SGX enclave and on an updated TCB version [13]. AMD SEV-SNP attestation reports include a launch measurement of initial guest memory and vCPU state that chains back to an AMD root of trust in cloud deployments such as AWS EC2 [14].

However, TEEs are evidence sources, not absolute authorities. They shift trust to vendors, firmware, microcode, attestation services, cloud management planes, and supply chains. Therefore:

- no single hardware root may dominate validator or admission authority;
- attestation should be ZK-wrapped or anonymized where privacy matters;
- critical actions require human and institutional quorums in addition to hardware evidence;
- TCB revocations must propagate rapidly across epochs;

- fallback paths must exist for open hardware, air-gapped recovery, and low-resource regions.

Direct Anonymous Attestation is relevant because it maps anonymous attestation into the RATS architecture, reducing the need to reveal a unique attester identity for every attestation event [12]. The clean epoch should ask: “prove you are an approved class of machine running approved code,” not: “identify your exact device to a global registry.”

10.3 Software Provenance

A rogue-AI scenario is more likely to exploit software supply chains than raw packet routing alone. TUF exists to protect software update systems even against repository or signing-key compromise [8]; its specification supports threshold and quorum trust to reduce key-compromise impact [9]. in-toto provides a framework for recording what supply-chain steps were performed, by whom, and in what order [10].

CONTINUITY WEB treats software provenance as epoch-critical. Executable authority should require:

- signed source and build provenance;
- threshold update roles;
- reproducible or independently rebuildable artifacts where practical;
- software bills of materials and dependency evidence;
- transparency-log inclusion;
- epoch compatibility metadata;
- rapid revocation and rollback;
- high-friction review for AI-authored or AI-modified critical code.

11 Stakeholder-Specific Participation Plan

Stakeholder	What they should build or contribute	Why it matters
White hats / red teams	Attack the prototype; design false-fork, poisoned-update, attestation-bypass, and AI-delegation capture exercises.	The design must survive adversarial testing before institutional adoption.
Cypherpunks / cryptographers	Design privacy-preserving admission, no-trusted-setup ZK circuits, nullifier schemes, anonymous attestation, and anti-censorship governance.	The system must avoid becoming surveillance infrastructure.

Universities / professors	Formalize threat models, prove invariants, study split-brain resolution, test HCI under crisis, and evaluate governance capture.	The concept spans distributed systems, cryptography, security, law, and social science.
Senior developers	Build reference clients, policy DSLs, manifest parsers, proof verifiers, quarantine services, SDKs, and agent gateways.	The idea becomes real only through interoperable tooling.
Government officials	Fund open standards, coordinate critical infrastructure pilots, define legal recognition of succession manifests, and prevent central capture.	Public legitimacy and continuity law are essential in a real crisis.
Critical infrastructure operators	Maintain signed checkpoints, continuity ledgers, emergency operator roles, and sector-specific quarantine policies.	Banks, hospitals, utilities, and public records cannot be restored like ordinary files.
Cloud and chip vendors	Provide interoperable attestation, transparent TCB revocation, open verifier tooling, and vendor-diversity guarantees.	Hardware evidence is useful only if auditable and non-monopolistic.
Funders / philanthropies	Support open research, reference implementations, audits, fellowships, and crisis drills.	The common-good components will not emerge from narrow commercial incentives alone.

12 Implementation Roadmap

12.1 Phase 0: Consortium and Specification

Duration target: 6 to 12 months.

Deliverables:

- public threat model and principles;
- wire format for Epoch Objects, Clean Genesis Manifests, and Succession Stamps;
- policy DSL for admission, quarantine, AI delegation, and quorum rules;
- reference governance model;
- initial legal analysis for continuity recognition;
- red-team scenario catalog.

12.2 Phase 1: Bounded Prototype

Build a forkable network for a bounded community such as an open-source package registry, university consortium, research lab network, or municipal emergency communications pilot.

Minimum viable features:

- cryptographic identities and recovery guardians;
- content-addressed storage;
- signed append-only logs;
- TUF/in-toto-style update provenance;
- Clean Genesis Manifest generation;
- Succession Stamp ledger with provisional, justified, finalized, and contested states;
- basic admission tickets;
- old-epoch quarantine import;
- AI Capability Passport gateway;
- simulated fork drill.

12.3 Phase 2: ZK and Attestation Integration

- implement no-trusted-setup ZK admission proofs;
- integrate heterogeneous attestation verifiers;
- implement privacy-preserving attestation result proofs;
- add nullifier-based anti-Sybil controls;
- implement recursive proof aggregation for epoch manifests;
- audit circuits and verifier code.

12.4 Phase 3: Critical Infrastructure Pilots

- sector-specific continuity ledgers for healthcare, finance, utilities, public records, and emergency response;
- legal recognition tests for signed continuity manifests;
- tabletop exercises with public-sector and private-sector operators;
- cross-border interoperability tests;
- fallback transport drills using mesh, satellite, and offline bundles;
- regional split-brain and delayed-reconciliation drills using stamped fork histories.

12.5 Phase 4: Standardization and Deployment

- submit protocol components to appropriate standards bodies;
- establish public test networks;
- build browser/client integration;
- create independent verifier implementations;
- launch public bug bounties and formal verification challenges;
- publish annual succession drills and audit reports.

13 Evaluation Metrics

A prototype should be judged by measurable resilience outcomes.

Metric	Question
Fork recognition time	How quickly can honest clients recognize the legitimate successor epoch under attack?
Admission privacy	How much sensitive identity, hardware, software, and location data is revealed during admission?
False-fork resistance	Can adversaries convince users or clients to join a counterfeit successor?
Succession-stamp finality	Can clients distinguish observed, justified, finalized, contested, and quarantined succession claims under partition?
Old-authority rejection	Can old credentials, AI delegations, update channels, or governance signatures command the new epoch?
Data recovery rate	How much useful old-epoch data can be safely recovered through quarantine?
AI containment	Can an AI agent exceed its passport, delegate recursively, spend beyond budget, or survive succession?
Attestation diversity	How much authority is concentrated in a single vendor, cloud, TCB, or jurisdiction?
Proof performance	Can users and services generate and verify admission and compliance proofs with acceptable latency and cost?
Liveness under partition	Can regional cells operate locally and reconcile later?
Governance capture resistance	Can one stakeholder class launch, block, or dominate succession?
UX under stress	Can non-experts identify current epoch, quarantine status, and recovery instructions during crisis?

14 Open Research Questions

1. **Policy-circuit correctness.** How do we prove the ZK circuit captures the policy humans believe they wrote?

2. **Attestation privacy.** How can we balance anonymous attestation with accountable abuse investigation?
3. **Human legitimacy.** Which institutions should be recognized as emergency guardians, and how do we prevent political capture?
4. **Succession Stamp Protocol.** What finality, slashing, weak-subjectivity, anchoring, and scope rules best handle split-brain succession without creating a global kill switch?
5. **Split-brain reconciliation.** How should honest stamped forks merge after partitions without importing malicious authority?
6. **Agent observability.** What is the minimum evidence needed to prove AI policy compliance without exposing private prompts, data, or reasoning traces?
7. **Legal continuity.** Can a signed Clean Genesis Manifest be recognized for records, contracts, emergency operations, and public services?
8. **False recovery UX.** How do ordinary users distinguish legitimate recovery from AI-generated impostor instructions?
9. **Critical service state.** How do banks, hospitals, utilities, and registries checkpoint real-world state without creating privacy and surveillance hazards?
10. **Post-quantum transition.** Which identity, signature, and proof systems need post-quantum migration paths, and how are those migrations made epoch-safe?
11. **Economic incentives.** How are idle continuity systems funded and maintained before they are needed?

15 Risk Register and Mitigations

Risk	Failure mode	Mitigation
False forks	Attackers announce counterfeit clean epochs.	Pre-registered recovery keys, multi-channel announcements, public manifests, Succession Stamps, challenge windows, and client-side fork-choice rules.
Governance capture	One class of actors controls succession.	Quorum-of-quorums, diversity caps, public logs, civil oversight, and split authority.
Key theft	Compromised credentials cross into successor epoch.	Epoch-specific rekeying, hardware-bound keys, short-lived capabilities, behavioral checks, and rapid revocation.
TEE monoculture	A vendor flaw compromises admission or validators.	Heterogeneous attestation and hard caps on vendor/root influence.

Bad ZK policy	Proofs validate the wrong thing.	Public policies, formal verification, multi-implementation testing, circuit audits, and human-readable policy explanations.
AI social manipulation	AI convinces humans to authorize dangerous actions.	Multi-person approval, cooling-off periods, transparency logs, spend/compute/delegation limits, and separation of powers.
Old-data poisoning	Quarantine imports carry malware or authority.	Sandboxing, stripping authority, content addressing, sanitizer proofs, and re-signing under successor policy.
Authoritarian misuse	Safety system becomes censorship or identity-control infrastructure.	Separate safety from speech authority, preserve inspectable history, minimize identity requirements, and provide appeal/re-admission.
Liveness failure	DDoS, censorship, or partitions prevent coordination.	Local cells, scoped provisional stamps, asynchronous consensus, fallback transports, offline proof bundles, and delayed reconciliation.

16 Recommended Minimum Viable Demonstration

The first credible demonstration should be narrow enough to build but serious enough to expose core problems. Recommended target:

A forkable open-source package registry and secure collaboration network for a university/security-lab consortium.

Why this target works:

- software supply-chain attacks are realistic and high-impact;
- developers can understand provenance, revocation, and update authority;
- universities can provide independent validators and red teams;
- the prototype can include AI coding agents with constrained capabilities;
- old-registry imports can be tested through quarantine;
- a simulated compromise can be staged ethically.

Demonstration scenario:

1. Red team compromises the ordinary update channel and several service identities.
2. Sentinel logs detect supply-chain divergence and suspicious AI-agent delegation.
3. Network escalates from Yellow to Orange to Red.
4. AI delegations are suspended; high-risk authority changes freeze.
5. Governance quorums publish a Clean Genesis Manifest and a justified Succession Stamp.
6. Clients recognize the stamped successor epoch and reject unstamped false forks.
7. Developers join using admission tickets.
8. Old packages enter through quarantine bridges.
9. Compromised update credentials fail to command the successor registry.
10. After-action report measures fork time, false-fork resistance, admission privacy, and data recovery.

17 Policy and Legal Considerations

Public-sector participation is useful but dangerous if it becomes central control. Governments should not own the fork switch. Instead, they should:

- fund open research and public-interest implementations;
- recognize signed continuity manifests for regulated critical infrastructure;
- require continuity drills for high-risk sectors;
- support international interoperability;
- protect civil liberties and due process in admission, exclusion, and quarantine policy;
- avoid mandating a single identity provider, attestation vendor, or national trust root;
- create emergency legal procedures for continuity of records, contracts, healthcare, utilities, and public services.

18 Conclusion

CONTINUITY WEB is an attempt to make digital civilization forkable before a network-wide legitimacy collapse. The central threat is not simply that attackers may alter data. It is that autonomous systems may contaminate the mechanisms by which society decides what is authentic, authorized, updated, and governed.

The proposed answer is not a global blockchain and not a single Blackwall. It is a constitutional trust-succession protocol with a narrow stamping layer for succession claims:

When the current epoch becomes untrustworthy, legitimate participants can reconstitute authority under a successor epoch, preserve useful data through quarantine, revoke broad AI agency, and continue operating without importing the compromised command structure of the old network.

The technology is plausible because its pieces exist. The system does not yet exist because the pieces have not been unified around this specific goal. The immediate task is to bring together cryptographers, white hats, systems researchers, senior engineers, public officials, infrastructure operators, and civil-liberties stakeholders to build a constrained prototype, attack it, and turn the surviving pieces into open standards.

The aim is not perfection. The aim is continuity: preserving human-recognizable legitimacy when the network environment itself can no longer be trusted.

A Appendix A: Glossary

Term	Definition
Authority	The ability to authenticate, sign, update, route, delegate, govern, or command within an epoch.
Trust epoch	A bounded policy and state universe defining legitimate identities, software, agents, governance, routing, and attestation roots.
Trust succession	The planned migration of authority from a compromised epoch to a successor epoch.
Clean Genesis Manifest	Public birth certificate of a successor epoch.
Succession Stamp Protocol	A narrow checkpoint/finality layer that orders, scopes, contests, finalizes, anchors, and reconciles succession claims.
Succession Stamp	A signed commitment to a manifest hash, evidence root, checkpoint, quorum root, validator set, scope, and status.
zk-attested admission ticket	A privacy-preserving proof that a node satisfies the successor epoch's admission policy.
AI Capability Passport	Scoped, expiring authority granted to an AI agent through a responsible principal.
Quarantine bridge	A controlled import path from old-epoch data into the successor epoch that strips authority.
Nullifier	A context-specific value that prevents double-use of a credential without revealing the credential itself.
TEE	Trusted execution environment; a hardware-backed protected execution context such as SGX, TDX, SEV-SNP, or related designs.
RATS	Remote ATtestation procedureS; an IETF architecture for attestation roles and evidence flows.
Split brain	The existence of multiple competing successor epochs claiming legitimacy.
Weak-subjectivity checkpoint	A recent trusted checkpoint or stamp obtained through independent channels, used to avoid being tricked by fabricated histories after long isolation.

B Appendix B: Minimal Data Model

```
EpochObject = {
  "epoch_id": string,
  "parent_epoch_id": string | null,
  "policy_hash": digest,
  "identity_root": merkle_root,
  "software_root": merkle_root,
  "attestation_root": merkle_root,
  "ai_capability_root": merkle_root,
  "namespace_root": merkle_root,
  "routing_policy_root": merkle_root,
  "revocation_root": merkle_root,
  "quarantine_policy_hash": digest,
  "governance_rule_hash": digest,
  "transparency_log_roots": [merkle_root],
}
```



```
"valid_from": timestamp,
"valid_until": timestamp,
"signatures": [signature]
}

SuccessionStamp = {
  "stamp_id": string,
  "parent_epoch_id": string,
  "parent_stamp_id": string | null,
  "event_type": string,
  "scope": string,
  "checkpoint_root": merkle_root,
  "clean_genesis_manifest_hash": digest,
  "evidence_bundle_root": merkle_root,
  "governance_quorum_root": merkle_root,
  "validator_set_root": merkle_root,
  "revoked_authority_root": merkle_root,
  "expired_ai_delegation_root": merkle_root,
  "quarantine_policy_hash": digest,
  "status": string,
  "timestamp_interval": [timestamp, timestamp],
  "external_anchor_roots": [digest],
  "proof_bundle_hash": digest,
  "signatures": [signature]
}

AdmissionTicket = {
  "epoch_id": string,
  "policy_hash": digest,
  "capability_class": string,
  "risk_class": string,
  "nullifier": digest,
  "proof_system": string,
  "proof": bytes
}

AICapabilityPassport = {
  "passport_id": string,
  "agent_id": string,
  "responsible_principal_commitment": digest,
  "model_or_deployment_class": string,
  "allowed_tools": [string],
  "allowed_data_scopes": [string],
  "allowed_write_scopes": [string],
  "spend_limit": number,
  "compute_limit": number,
  "delegation_depth_limit": integer,
  "expiry": timestamp,
  "survives_epoch_transition": false,
  "policy_hash": digest,
  "issuer_signatures": [signature]
}
```

Minimal JSON-like data model

C Appendix C: Initial Work Packages

Work package	Output	Lead profile
WP1: Threat model	Rogue-AI, false-fork, supply-chain, TEE, governance-capture, and physical-partition scenarios.	Security researchers, red teams
WP2: Manifest spec	Canonical encoding, signatures, policy hashes, transparency-log commitments.	Protocol engineers
WP3: Policy DSL	Human-readable and machine-verifiable policy language for admission, quarantine, AI, and governance.	PL/formal methods teams
WP4: ZK admission	No-trusted-setup proof for identity continuity, non-revocation, attestation result, and nullifier.	Cryptographers
WP5: Attestation bridge	Multi-vendor RATS-compatible verifier abstraction and privacy-preserving result format.	Confidential computing engineers
WP6: AI gateway	Capability passport issuance, tool-gateway enforcement, logs, and compliance proofs.	AI safety/security engineers
WP7: Quarantine bridge	Old-object import, sanitizer policies, proof-carrying outputs, and audit logs.	Malware analysts, platform engineers
WP8: Succession stamps	Provisional, justified, finalized, contested, anchored, and reconciled stamp states; equivocation evidence; scoped fork-choice rules.	Distributed systems engineers
WP9: Governance drills	Fork ceremonies, tabletop exercises, legal continuity tests, false-fork UX.	Public-sector, civil society, HCI teams

D Appendix D: Bibliography

References

- [1] Ethereum.org. “Timeline of all Ethereum forks.” <https://ethereum.org/ethereum-forks/>. Accessed April 2026.
- [2] Ethereum.org. “Gasper.” <https://ethereum.org/developers/docs/consensus-mechanisms/pos/gasper/>. Accessed April 2026.
- [3] Ethereum.org. “Rewards and penalties.” <https://ethereum.org/developers/docs/consensus-mechanisms/pos/rewards-and-penalties/>. Accessed April 2026.
- [4] Ethereum.org. “Weak subjectivity.” <https://ethereum.org/developers/docs/consensus-mechanisms/pos/weak-subjectivity/>. Accessed April 2026.
- [5] IPFS Docs. “Content addressing and CIDs.” <https://docs.ipfs.tech/concepts/content-addressing/>. Accessed April 2026.

- [6] Certificate Transparency. “How CT Works.” <https://certificate.transparency.dev/howctworks/>. Accessed April 2026.
- [7] World Wide Web Consortium. “Decentralized Identifiers (DIDs) v1.0.” <https://www.w3.org/TR/did-core/>. Accessed April 2026.
- [8] The Update Framework. “A framework for securing software update systems.” <https://theupdateframework.io/>. Accessed April 2026.
- [9] The Update Framework. “TUF Specification.” <https://theupdateframework.github.io/specification/latest/>. Accessed April 2026.
- [10] in-toto. “A framework to secure the integrity of software supply chains.” <https://in-toto.io/>. Accessed April 2026.
- [11] Internet Engineering Task Force. “RFC 9334: Remote ATtestation procedureS (RATS) Architecture.” <https://www.rfc-editor.org/rfc/rfc9334.html>. 2023.
- [12] Internet Engineering Task Force. “Direct Anonymous Attestation for the Remote Attestation Procedures Architecture.” <https://datatracker.ietf.org/doc/draft-ietf-rats-daa/>. Accessed April 2026.
- [13] Intel. “Attestation Services for Intel Software Guard Extensions.” <https://www.intel.com/content/www/us/en/developer/tools/software-guard-extensions/attestation-services.html>. Accessed April 2026.
- [14] Amazon Web Services. “Attest an Amazon EC2 instance with AMD SEV-SNP.” <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/snp-attestation.html>. Accessed April 2026.
- [15] SCION Documentation. “SCION Overview.” <https://docs.scion.org/en/latest/overview.html>. Accessed April 2026.
- [16] Zcash. “NU5 Upgrade.” <https://z.cash/upgrade/nu5/>. Accessed April 2026.
- [17] Zcash Improvement Proposals. “ZIP 224: Orchard shielded protocol.” <https://zips.z.cash/zip-0224>. Accessed April 2026.
- [18] Semaphore Documentation. “What is Semaphore?” <https://docs.semaphore.pse.dev/>. Accessed April 2026.
- [19] National Institute of Standards and Technology. “SP 800-207: Zero Trust Architecture.” <https://csrc.nist.gov/pubs/sp/800/207/final>. 2020.
- [20] Miguel Castro and Barbara Liskov. “Practical Byzantine Fault Tolerance.” Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI), 1999. <https://pmg.csail.mit.edu/papers/osdi99.pdf>.